

Project Report: Finance Chatbot for Indian Stock Market

Team Members: Himanshu Dotsara

Abstract

This project report contains the details on our chatbot project which aims to provide informed, evidence-based responses to queries about the Indian stock market. It handles both general sector inquiries and detailed company analyses, considering real-time global events impacting the market. Data was collected through web scraping and labeled for training, with existing pretrained models adapted for efficient performance within memory constraints. The chatbot's interface enables user interaction, offering transparent responses backed by contextual evidence. Deliverables include the trained model, chatbot interface, performance metrics, and a comprehensive report detailing the project phases, experiments, and methodologies.

1 Introduction

The Indian stock market, shaped by both local and global events, requires quick, data-backed insights for informed trading decisions. This project develops a conversational chatbot that addresses specific and general queries about market trends, sectors, and companies, with responses grounded in recent data and clear explanations.

Our approach includes collecting up-to-date data through web scraping, labeling, and augmentation, as well as adapting pretrained models for this task. A custom neural network architecture was designed to manage query complexity efficiently. Deliverables include a trained model, a chatbot interface, and a detailed report on each development stage, providing a valuable resource for Indian market insights.

We provide a survey of existing literature in Section 3. Our proposal for the project is described in Section 4. We give details on experiments in Section 6. A description of future work is given in Section 8. We conclude with a short summary and pointers to forthcoming work in Section 9.

2 Project Workflow

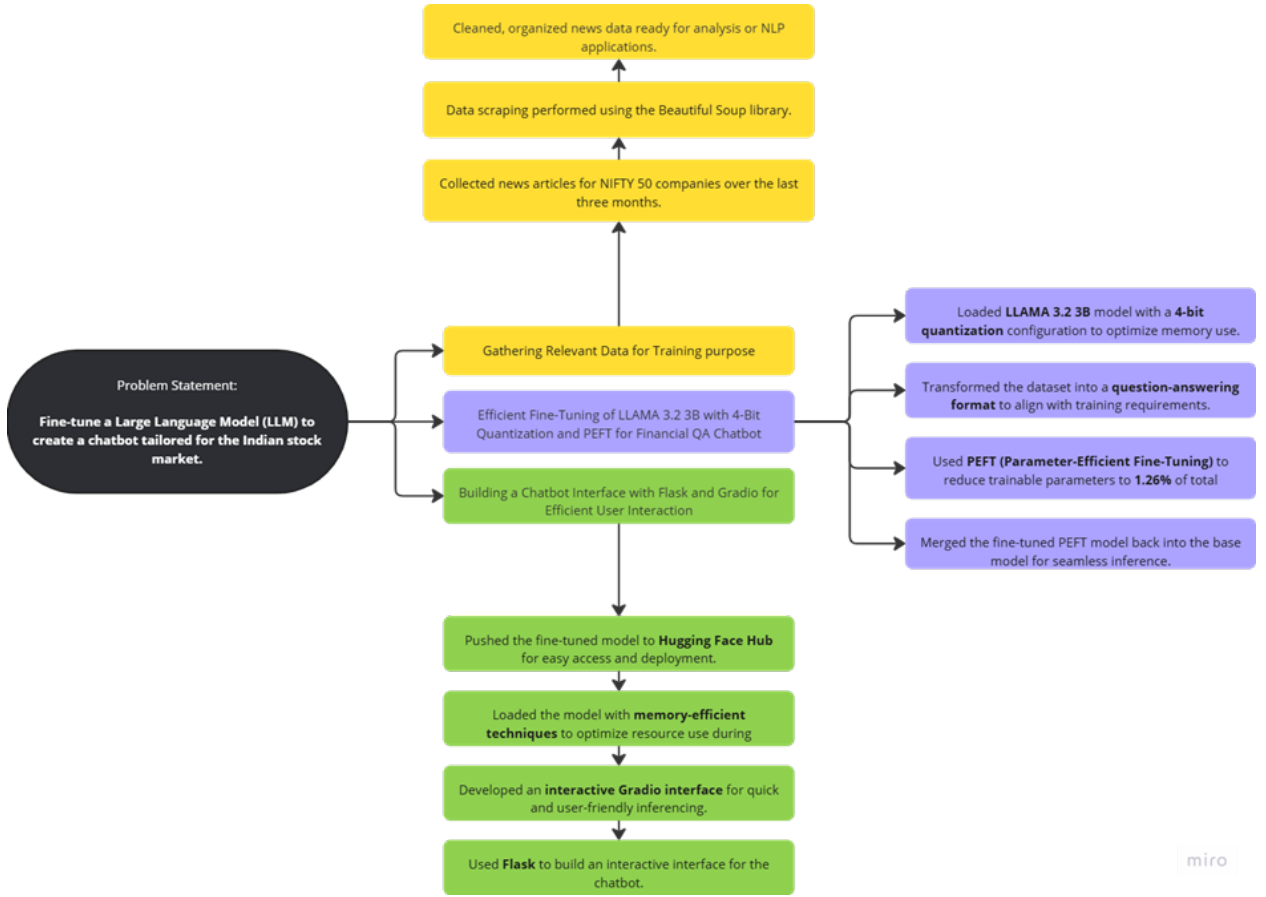


Figure 1: Project Workflow

3 Literature Survey

Our fine-tuning approach draws from methodologies outlined in three influential research papers: *"Empirical Insights on Fine-Tuning Large Language Models for Question-Answering"* provides guidance on optimizing LLMs specifically for QA tasks, which informed our data processing and model adjustment strategies. From *"QLoRA: Efficient Fine-Tuning of Quantized LLMs,"* we adopted quantization techniques to enhance memory efficiency and computational manageability without compromising model performance. Finally, insights from *"LLaMA: Open and Efficient Foundation Language Model"* supported our model selection, leveraging LLaMA's scalable architecture to achieve an optimal balance between size and efficiency for QA fine-tuning.

3.1 Empirical Insights on Fine-Tuning Large Language Models for Question-Answering

This part of our project draws inspiration from a closely related work by Junjie Ye and others.[1]. The research paper examines fine-tuning strategies for large language models (LLMs) in question-answering (QA)

LLM fine-tuning at a high level

LLM fine-tuning

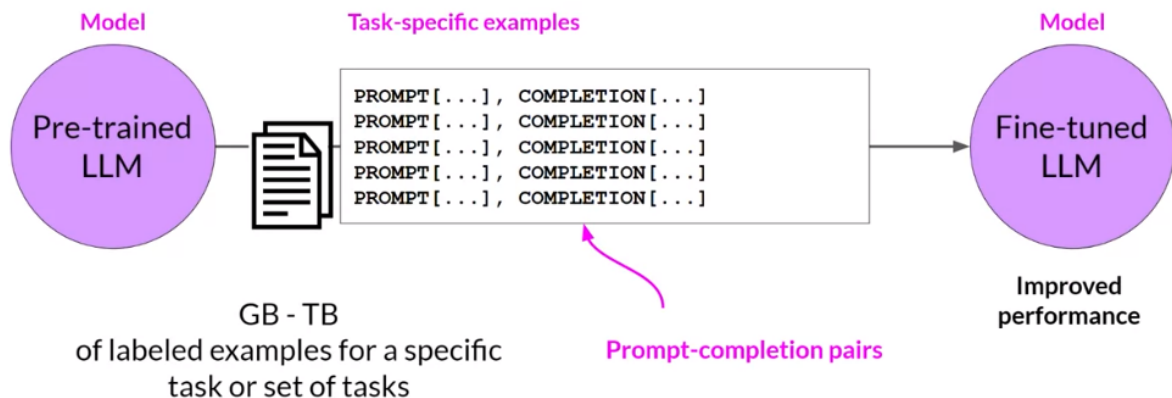


Figure 2: Finetuning an LLM for specific task

applications, emphasizing the challenges and impact of knowledge memorization on model performance. Large language models leverage vast, diverse pre-training datasets, but fine-tuning them for QA often encounters issues with memory retention and dataset effectiveness. To address these challenges, the study investigates how varying data volumes and memory characteristics influence fine-tuning success across different LLM families, particularly within the LLaMA and Qwen models.

Three primary research questions guide this analysis: determining the necessary data volume for effective fine-tuning, assessing how dataset choice influences performance, and exploring how these requirements differ across LLM architectures. The researchers employ a "multi-template complementation mechanism" to assess memory retention, using a dataset with 24 topics derived from Wikipedia, organized into in-domain, out-of-domain, and memory-specific sets. Findings reveal that effective QA fine-tuning can occur with as few as 60 data samples, activating pre-trained knowledge without requiring extensive data additions, which can sometimes reduce performance. High-memory-level data improves model responses, while low-memory-level data may impair accuracy, highlighting a "diagonal effect" where models perform optimally on memory-aligned data.

3.2 LLaMA: Open and Efficient Foundation Language Models

This part of our project draws inspiration from a closely related work by Tim Dettmers and others. [2]. The paper introduces LLaMA (Large Language Model Meta AI), a collection of foundational language models ranging from 7B to 65B parameters, designed with a focus on efficiency, accessibility, and competitive performance. LLaMA models employ an architecture based on transformers with modifications aimed at optimizing training efficiency and model stability. Key adaptations include pre-normalization layers, SwiGLU activations, and rotary embeddings to enhance positional encoding—a combination shown to stabilize training across models of varying sizes.

Instruction Finetuning

LLaMA's performance further improves through instruction finetuning, which enhances its ability to generalize across diverse language understanding tasks, particularly in the Massive Multitask Language Understanding (MMLU) benchmark. The model's performance gains from minimal finetuning suggest high adaptability, making it a promising candidate for applications requiring general language understanding.

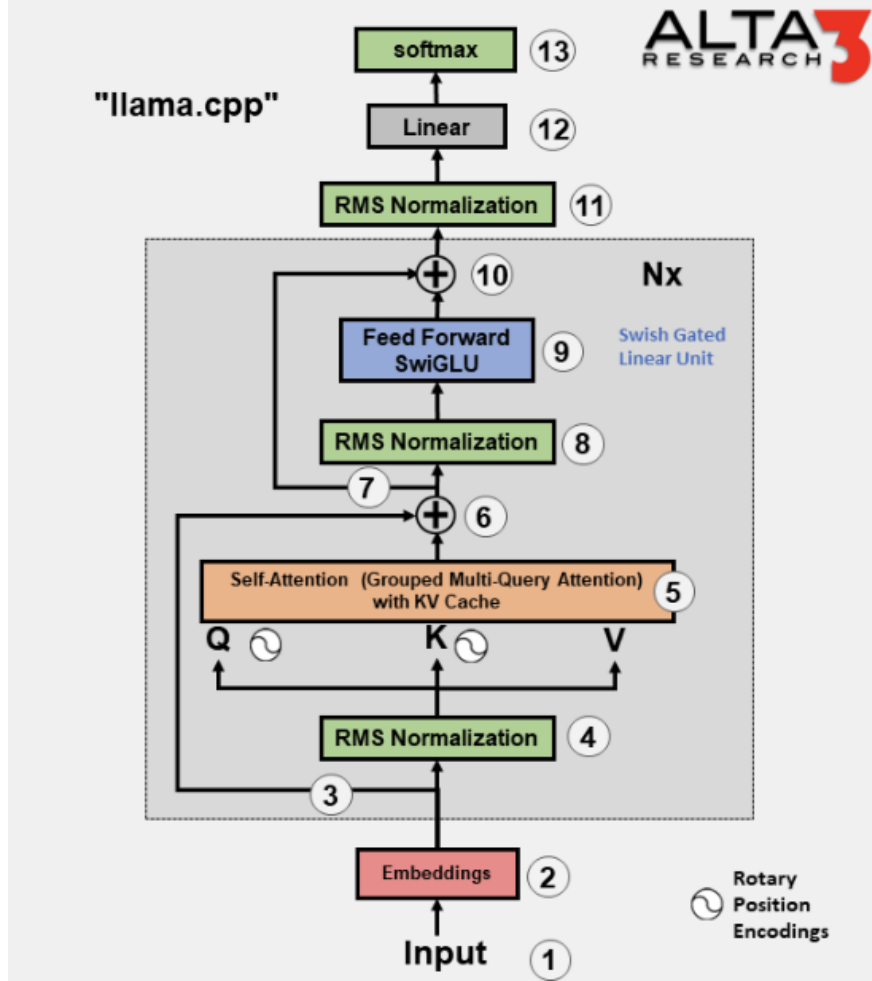


Figure 3: Project Workflow

A comprehensive assessment of LLaMA's energy consumption reveals a relatively low carbon footprint, attributed to the model's efficient design and optimized training protocols. LLaMA's smaller models, in particular, offer high performance at reduced computational costs, facilitating deployment on single GPUs and supporting research sustainability goals by lowering energy demands for comparable results.

3.3 QLORA: Efficient Finetuning of Quantized LLMs

This part of our project draws inspiration from a closely related work by Hugo Touvron and others. [3]. In their paper [2], the authors describe that QLORA is a fine-tuning approach designed to make fine-tuning large language models (LLMs) highly memory-efficient. By quantizing LLMs to 4-bit precision, QLORA enables fine-tuning models as large as 65 billion parameters on single 48GB GPUs. This method performs

as well as traditional 16-bit fine-tuning, making it feasible for a broader range of users without extensive computational resources.

4-bit NormalFloat (NF4): QLoRA uses NF4, a quantization format optimized for normally distributed weights. Unlike traditional 4-bit data types, NF4 preserves more information by evenly distributing quantization levels across normally distributed values, reducing quantization error, especially for outliers.

Double Quantization: This approach reduces memory usage by applying an additional quantization layer to the quantization constants themselves. For example, by quantizing these constants to 8-bit values, the memory needed for storing these constants is significantly reduced.

Paged Optimizers: Paged optimizers handle memory overflow during gradient checkpointing, which traditionally spikes memory usage. By using NVIDIA’s unified memory, QLoRA efficiently handles memory-intensive tasks like processing long sequences without causing out-of-memory errors.

Benchmarking and Evaluation

QLoRA’s effectiveness was tested with over 1,000 models across different datasets and model types, including LLaMA, T5, and model sizes from 80M to 65B parameters. The study showed that high-quality instruction datasets (e.g., OASST1) yield better performance than larger but less precise datasets.

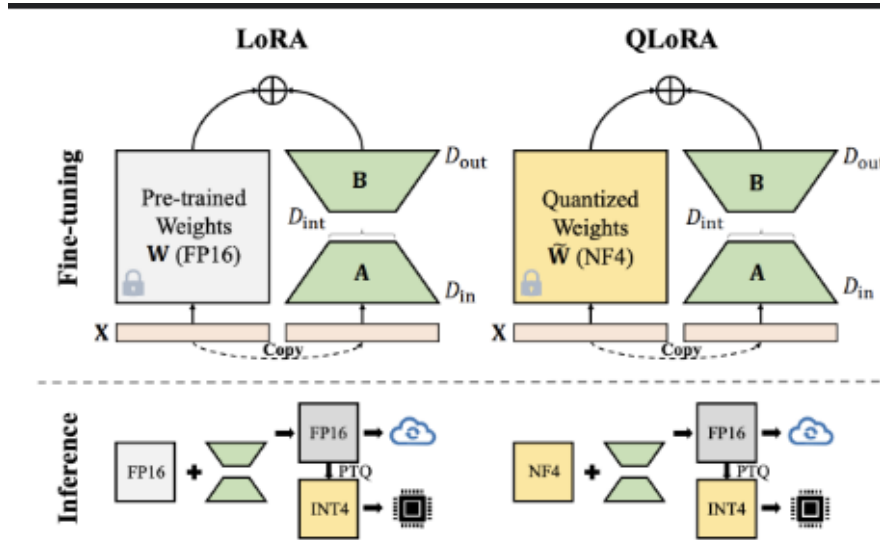


Figure 4: 4 bit quantization

4 Proposed Approach or Approaches

The core architecture and foundation of our model is grounded in Meta’s Llama-3.2-3B-Instruct, a robust 3-billion-parameter model tailored specifically for handling instructional tasks. This causal language model (CLM) architecture is designed to generate text in a sequential manner, predicting the next token based on the preceding sequence. The model itself is an instruction-tuned variant of Meta’s Llama series, optimizing it to excel at tasks where following and responding to explicit instructions is paramount.

Novel Aspects and Differentiating Features

Quantization Strategy: The model adopts a memory-efficient 4-bit quantization approach utilizing Bit-

sAndBytes, which includes specific configurations like 4-bit load optimization, double quantization, and FP4 quantization type. This carefully chosen quantization setup significantly reduces memory usage without sacrificing performance.

Parameter-Efficient Fine-Tuning (PEFT):

Leveraging LoRA (Low-Rank Adaptation) for fine-tuning, the model focuses on enhancing efficiency by updating only a targeted subset of parameters. Key settings include a rank of 32, scaling factor (alpha) of 32, and targeted modules such as `q_proj`, `o_proj`, and `v_proj`, among others, with a 0.1 dropout rate. This configuration is optimized for question-answering tasks, allowing for fine-tuning with minimal adjustments to the base model, which remains frozen.

Our approach stands out from previous models through a unique blend of memory efficiency and precision-driven optimization, setting new standards for resource use in large language model training. We implemented a 4-bit quantization combined with LoRA, making it possible to fine-tune a large 3-billion-parameter model on consumer hardware—a task typically reserved for high-end systems. Notably, we optimized the process by using only 53% of available data sources, managing a total of just 38 million trainable parameters, which dramatically reduced the memory footprint without sacrificing effectiveness. Unlike general-purpose models, our custom LoRA configuration specifically targets key transformer modules, optimizing the model for financial question-answering tasks. To further enhance training stability, we incorporated techniques like early stopping, gradient accumulation to enable effective larger batch sizes, and a linear learning rate schedule. This blend of data-efficient training, memory-conscious design, and tailored optimization results in a more accessible and highly specialized model.

ARCHITECTURE OF LLAMA 3:

EMBEDDINGS:

Embeddings convert discrete text inputs (like words or subwords) into continuous numerical vectors. Each word or token in the vocabulary gets assigned a unique embedding vector. For example, the word "economy" might have an embedding like $[0.25, -0.34, 0.89, \dots]$, where each number in the vector holds specific meaning in the model's high-dimensional space. In Llama, this transformation occurs at the very first layer, enabling text inputs to become mathematically interpretable.

RMS NORMALIZATION:

After tokens are transformed into dense vector embeddings, RMS normalization is applied to these embeddings to normalize the magnitude of each vector. This step ensures that all tokens, regardless of their initial embedding values, have a similar scale when entering the transformer layers. RMS normalization thus reduces the risk of large fluctuations in input values, which can destabilize the early stages of processing.

ROTARY POSITIONAL ENCODING:

Rotary Positional Encoding encodes the position of each token by rotating token embeddings in a multi-dimensional space. Instead of adding fixed positional embeddings to token embeddings (like sinusoidal or learned positional encodings), RoPE modifies the embedding itself by rotating it according to its position in the sequence. This technique preserves positional relationships and enables the model to generalize better for longer contexts.

Traditional positional encodings require either fixed values or learned parameters, which add to the model's complexity and can increase memory usage. RoPE, by contrast, does not require any additional learnable parameters.

RoPE has a unique advantage in handling long sequences. Because it rotates embeddings rather than adding fixed values, it better retains information about the relative positions of tokens, even when sequences are extended. This property makes it useful in Llama, which needs to manage long text passages effectively.

MULTIHEAD SELF ATTENTION:

Llama utilizes multi-head self-attention, which allows the model to capture different types of relationships by learning multiple sets of attention weights. Each head independently computes the attention output using separate weight matrices, and the results are concatenated and projected back into the embedding space. Self-attention allows Llama to understand and generate contextually relevant text by considering all tokens in relation to each other, regardless of their position in the sequence.

FEED FORWARD LAYER WITH SwiGLU ACTIVATION:

In the Llama architecture, the **feed-forward layer** plays a crucial role in processing information after the self-attention mechanism, utilizing the **SwiGLU** (Swish-Gated Linear Unit) activation function to enhance expressiveness and performance. This layer consists of two linear transformations separated by the SwiGLU activation. Initially, the input embeddings or the output from the self-attention layer undergo a linear transformation to project them into a higher-dimensional space. The resulting values are then passed through the SwiGLU activation, which combines the smooth, non-monotonic characteristics of the Swish function with the selective focus of a gated linear unit. This allows the model to retain valuable features while effectively controlling information flow. Finally, the output is transformed back into the original embedding size through another linear layer.

$$\text{Swish}_\beta(x) = x\sigma(\beta x)$$

$$\sigma(z) \sim \text{sigmoid function}$$

(a) Swish Function

$$h_l(\mathbf{X}) = (\mathbf{X} * \mathbf{W} + \mathbf{b}) \otimes \sigma(\mathbf{X} * \mathbf{V} + \mathbf{c})$$

(b) GLU Function

This activation function has been empirically shown to improve the performance of various models, especially in natural language processing tasks, making it a valuable choice for modern architectures such as BERT, GPT, and LLAMA.

Applying Softmax after RMS Normalization:

In LLAMA, the typical workflow involves processing data through several layers of transformations. After the final layer of the model (such as a feed-forward neural network or a multi-head attention layer), you would often apply RMS normalization to the output, followed by the Softmax function. The normalized logits are then passed through the Softmax function. This converts the logits into a probability distribution over the possible output classes or tokens, making it suitable for tasks like language generation or classification.

4.1 Work done before prep-presentation review

4.1.1 Data Collection

Companies and Data Source:

- **Companies:** Includes 50 major Indian companies like Reliance Industries, HDFC, Tata Steel, and Infosys.
- **Source:** Moneycontrol, using structured URLs to access articles tagged with company names. Two pages of articles were retrieved per company using pagination.

Process Overview:

- **Libraries Used:** requests, BeautifulSoup, pandas, json.
- **URL Construction and Scraping:** Articles were collected by constructing URLs dynamically and parsing the HTML content.
- **Content Extraction:** Relevant article text was extracted from `<p>` tags, and data for each company was saved in JSON format (e.g., `reliance-industries.json`).

4.1.2 Data Cleaning

To improve data quality, the text was processed to remove irrelevant content and special characters:

- **Cleaning Steps:** Disclaimers, unwanted phrases, and unnecessary punctuation were removed.
- **Storage:** Cleaned JSON files were generated, preserving structured and high-quality data.

4.1.3 QA Pair Generation from News Articles

1. **Prompt Creation for QA Generation:** We designed a prompt to guide the language model in generating QA pairs from each paragraph in the news articles. Each generated QA pair was structured to include:
 - **Question:** A question relevant to the company or financial topic discussed in the paragraph.
 - **Answer:** A concise answer derived from the information presented in the paragraph.
2. **Model Configuration and Execution:** Using OpenAI's Gemini model through the g4f client API, we set specific parameters to optimize the QA generation process:
 - **Output Format:** Each QA pair was generated as a dictionary with keys for `question` and `answer`.
 - **Batch Size:** Eight QA pairs were generated for each paragraph to maximize information extraction while keeping redundancy minimal.
 - **Formatting Constraints:** The prompt included strict guidelines to ensure output consistency, such as generating all outputs in list-of-dictionaries format.
3. **Automated Iterative Process:**

- For each company's news articles, we processed each paragraph individually to generate distinct QA pairs.
 - The language model was invoked with the prepared prompt and paragraph content, resulting in a set of eight unique QA pairs for each article segment.
 - The output QA pairs were appended to a list, organized for easy integration into the overall dataset.
4. **Saving the Generated Data:** Each QA pair list was saved as a JSON file, systematically organized by company. This provided a structured and easily accessible dataset that could be readily used for chatbot training.

4.2 Work done after prep-presentation review

4.2.1 Increasing Dataset Size for Improved Model Generalization and adding the context key

To enhance the performance and generalization capabilities of the chatbot, we have significantly increased the dataset size from 10,000 to 17,000 question-answer pairs. Large language models (LLMs) thrive on extensive and diverse datasets, which enable them to learn more robust representations of language and context. By augmenting the dataset, we provide the model with a broader spectrum of financial queries and responses, allowing it to capture intricate patterns and relationships in the data. Including context with the question and answer allows the model to better understand the specific scenario or information being queried. The context provides background information that is essential for accurate interpretation, enabling the model to respond more effectively to user inquiries.

4.2.2 Implementing 4-Bit Quantization with QLoRA for Efficient Memory Management

To optimize memory usage and enhance the efficiency of our chatbot model, we have implemented the 4-bit quantization technique known as QLoRA (Quantized Low-Rank Adaptation). This method reduces the memory footprint of the large language model (LLM) while maintaining its performance during training and inference. By quantizing the model weights from their original 32-bit floating-point representations to 4 bits, we significantly decreased the amount of memory required to store and process the model.

4.2.3 Merging the Trained PEFT Model with the Base Model for Inference and Deployment on Hugging Face Hub:

The PEFT approach focuses on optimizing only a small subset of parameters, rather than fine-tuning the entire model. This allows us to maintain the original capabilities of the base model while enhancing its performance on specific tasks, such as answering questions related to the Indian stock market. By merging the PEFT model with the base model, we create a single, cohesive model that benefits from the efficiency of fine-tuning while preserving the knowledge encoded in the larger architecture. This deployment strategy enhances the model's accessibility, allowing it to be seamlessly integrated into various applications and workflows. Additionally, hosting the model on Hugging Face provides robust version control.

4.2.4 Creating Two Separate Interfaces for the Chatbot Using Flask and Gradio:

In our project, we utilized Flask and Gradio to create two distinct interfaces for the chatbot, each serving a specific purpose and enhancing user experience. Users can choose between the Flask interface for more complex interactions or the Gradio interface for a simplified experience.

5 Data set Details

1. Dataset Description

- **Content:** The dataset consists of a collection of question-answer pairs focused on the Indian stock market, specifically addressing inquiries related to companies, brokerages, and financial forecasts.
- **Size:** The dataset currently includes **17000 entries**. As the project progresses, additional entries will be gathered to enhance the model's performance.

2. Attributes The dataset contains the following attributes:

- **Question:**
 - **Type:** Text (String)
 - **Description:** The inquiry posed by the user regarding financial metrics or stock performance.
 - **Example:** "What is the expected revenue growth of Emcure Pharma according to Jefferies?"
- **Answer:**
 - **Type:** Text (String)
 - **Description:** The response to the question, providing specific insights or data points.
 - **Example:** "11% revenue CAGR from market share gains in India."
- **Context:**
 - **Type:** Text (String)
 - **Description:** Additional background information that contextualizes the question and answer, enhancing the understanding of the financial analysis.
 - **Example:** "Initiate With A Buy Call Target At Rs1600 Have Strong Chemistry Skills In Developing Complex Molecules."

```

{
  "question": "What is the category of Tata Steel Ltd?",
  "answer": "The category of Tata Steel Ltd is Metals major.",
  "context": "Metals major Tata Steel Ltd reported a 51 percent jump in consolidated net profit at Rs960 crore
},
{
  "question": "What is the nature of the change in consolidated net profit of Tata Steel Ltd?",
  "answer": "The nature of the change in consolidated net profit of Tata Steel Ltd is a jump.",
  "context": "Metals major Tata Steel Ltd reported a 51 percent jump in consolidated net profit at Rs 960 cro
},
{
  "question": "How much might Tata Steel need to pay as minerals tax dues to Odisha?",
  "answer": "More than Rs 17000 crore",
  "context": "Tata Steel might need to pay more than Rs17000 crore as minerals tax dues to the state of Odish
},

```

Figure 6: dataset example

3. Nature and Type of Data

- **Nature:** The dataset comprises qualitative data, specifically in the form of textual information.
- **Type:** The data type is purely text, with no images, audio, or video components involved.

Data Source: The data for this project has been gathered from **Moneycontrol.com**, a prominent financial news website that provides comprehensive information on stocks, market trends, and corporate announcements. Specifically, the dataset consists of news articles related to **NIFTY 50** companies, covering their financial performance, analyst reports, and market predictions.

Generating Question-Answer Pairs from News Articles Using GPT-4-Free Client and LLAMA 3.1 Model:

To generate question-answer pairs from news articles, we used a script that combines the GPT-4-Free client with the LLAMA 3.1 405B model. The process starts by scraping news articles about specific companies to gather relevant content. We then define a function, `qa_pair_generator`, which takes a paragraph of text as input and constructs prompts for the model to generate eight question-answer pairs formatted as dictionaries. These prompts ensure that company names are included and that the context is both descriptive and adheres to length requirements. After sending the request to the model, we receive a JSON output containing structured data with each question, its answer, and the relevant context. This automated method efficiently creates a question-answer dataset, making it valuable for training chatbots and improving financial information retrieval systems.

Preprocessing Question-Answer Pairs for Model Input:

To transform the generated question-answer pairs into a format suitable for model training, we utilized a preprocessing function that effectively organizes and tokenizes the data. Inside the `preprocess_function`, we prepare the input by combining each question with its corresponding context using a formatted string: `"Question: {q} Context: {c}"`. This allows the model to understand the relationship between the question and the provided context. The outputs, which are the answers, are stored separately. We then tokenize both the inputs and outputs. Finally, we set the `input_ids` of the labels(answers) in the `model_inputs` dictionary, which will be used during the training phase.

6 Experiments

This report outlines the experiments conducted to fine-tune various language models on a custom financial question-answering (QA) dataset. Each experiment aimed to optimize the model's ability to generate accurate and contextually relevant responses to financial queries. We explored different approaches, including dataset enhancements, model quantization, retrieval-augmented generation (RAG), and fine-tuning techniques such as LoRA and QLoRA.

6.0.1 Dataset Preparation and Model Training

Objective : To create an initial dataset and fine-tune a financial chatbot that generates accurate and contextually relevant answers to financial questions.

Process

1. **Initial Dataset Creation:** We began with 10,000 question-answer pairs sourced from financial news articles. These pairs covered financial trends, companies, and market sector insights, with answers either extracted or inferred from article content.
2. **Context Enhancement:** The initial dataset lacked context, leading to inaccurate responses during testing. To improve this, we transformed each question-answer pair into a triplet by adding a *context field* that provided additional information to clarify or support each answer.
3. **Model Quantization:** To reduce memory usage, the model was quantized to 4-bit precision. This adjustment introduced some performance limitations due to the dataset's size, which impacted the model's learning capacity.
4. **Dataset Expansion:** To counteract the limitations from quantization, we expanded the dataset to 17,000 question-answer-context triplets, which enhanced data diversity and the model's ability to handle a broader range of financial queries.

Outcome: The quantized 4-bit model trained on the 17,000 question-answer-context pairs demonstrated improved response accuracy, achieving a balance between memory efficiency and response quality.

6.0.2 Retrieval-Augmented Generation (RAG) Application Using LlamaIndex

Objective To test a RAG-based chatbot that combines retrieval and generation capabilities, allowing responses to be generated from indexed documents on specific financial topics.

Setup

1. **Model Loading:** The Llama2 model was loaded using the Ollama API, configured for local use to test its response accuracy with specific query contexts (e.g., Mukesh Ambani's visit).
2. **External Document Integration:**
 - *Data Sources:* Financial and technology-related content was collected from various URLs.

- *Processing*: Content was cleaned using `requests` and `BeautifulSoup` for compatibility with LlamaIndex.
 - *Storage*: Documents were formatted for LlamaIndex, enabling quick retrieval.
3. **Index Building**: LlamaIndex's `VectorStoreIndex` with Hugging Face embeddings (BAAI/bge-small-en-v1.5) enabled semantic search for retrieving relevant documents.
 4. **Pipeline Construction**: The RAG pipeline linked Llama2 with the document index, allowing complex queries to retrieve top documents and generate context-based responses.

Outcome: While the model answered document-specific questions accurately, it relied heavily on exact matches within indexed documents. This dependency limited its ability to answer broader financial questions, making it unsuitable as the primary model for a financial chatbot, which requires a comprehensive knowledge base.

6.0.3 Fine-Tuning Llama-2-7B with LoRA and QLoRA

Objective: To fine-tune the Llama-2-7B-chat model using LoRA and QLoRA for efficient, domain-specific responses within a memory-constrained environment.

Process

1. Configuration:

- *LoRA Attention Dimension*: Set to 64.
- *Alpha Parameter*: 16 for scaling.
- *Dropout Probability*: 0.1.
- *Quantization*: 4-bit QLoRA enabled for memory efficiency.

2. Training:

- *Optimizer*: Paged AdamW with 32-bit weights.
- *Scheduler*: Cosine learning rate scheduler with a 3% warm-up.
- *Learning Rate*: $2e-4$.
- *Batch Size*: 4 (limited by the available GPU).

3. Hardware: GPU: NVIDIA T4 (15GB memory) on Google Colab, with fp16 precision.

Outcome: The quantized and LoRA-configured model faced issues with response accuracy and convergence due to limited batch size and gradient accumulation steps. The T4 GPU's constraints limited performance, and the model failed to deliver consistent domain-specific accuracy.

6.0.4 Fine-Tuning Meta LLaMA-3 8B for Financial QA

Objective To fine-tune Meta's LLaMA-3 8B Instruct model for financial question-answering using LoRA and Flash Attention (Xformers) on a constrained hardware setup.

Process

1. Configuration:

- *Model and Tokenizer*: Meta LLaMA-3 8B Instruct model with right-padding to avoid overflow.
- *LoRA Parameters*: Set with an attention rank of 16 and a scaling factor of 16.
- *Flash Attention*: Enabled for efficient training.

2. Optimization:

- *Optimizer*: AdamW with 8-bit precision.
- *Learning Rate*: $2e-4$ with linear scheduling.
- *Batch Size*: 2 with gradient accumulation steps set to 4.

3. Hardware: GPU: NVIDIA T4 (15GB memory) with fp16 precision on Google Colab.

Outcome: LoRA and Flash Attention accelerated training, but the model's response quality suffered due to small dataset size and memory constraints. Although the fine-tuning process was successful, resource limitations restricted the model's potential for optimal accuracy in financial QA.

6.0.5 Fine-Tuning Meta LLaMA-3.2 3B INSTRUCT for Question Answering

Objective: To fine-tune the Meta LLaMA-3.2 3B model for QA using LoRA and BitsAndBytes quantization, optimizing memory usage on limited hardware resources.

Process

1. Configuration:

- *Model and Tokenizer*: Meta LLaMA-3.2 3B, using AutoModelForCausalLM and pad.token set to eos_token.
- *LoRA Parameters*: Rank set to 32, with a scaling factor of 32 and dropout of 0.05.
- *Quantization*: Enabled 4-bit quantization with BitsAndBytes and float16 compute type for GPU compatibility.

2. Optimization:

- *Batch Size*: Training batch size of 16 with gradient accumulation set to 4.
- *Learning Rate*: $5e-5$ with linear scheduling.

3. Hardware: GPU: NVIDIA Tesla P100 (16GB memory) on Kaggle, using fp16 precision.

Outcome: The fine-tuning process, enhanced by LoRA and 4-bit quantization, was efficient in the Kaggle environment. The model demonstrated improved validation accuracy, though larger batch sizes and more epochs would be required to maximize performance. Future iterations may benefit from increased computational resources.

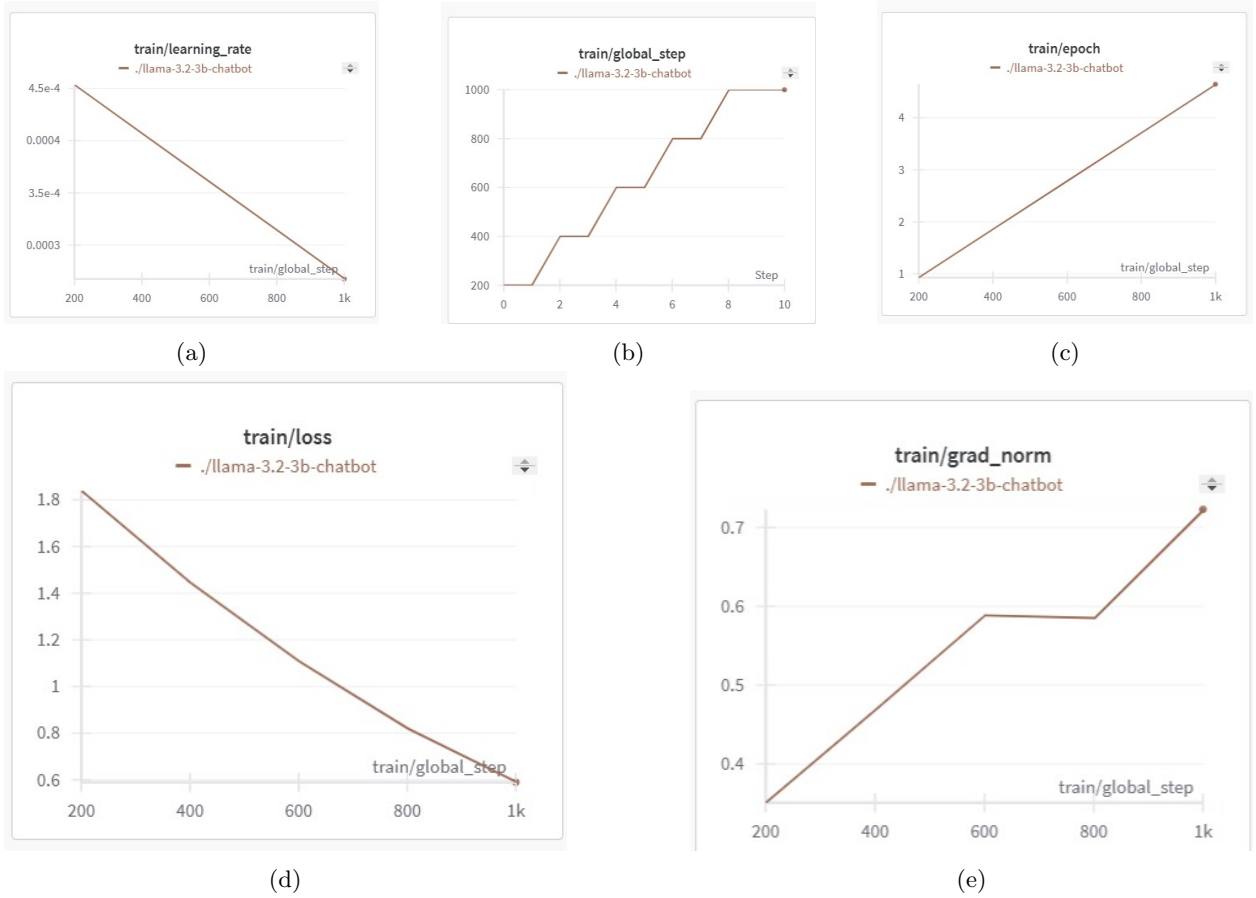


Figure 7: Training Metrics of Fine-Tuned Model Across Global Steps

7 Results

Description: The charts shown in figure 6 illustrate various training metrics for the fine-tuned financial chatbot model across global steps. These metrics provide insights into the model’s learning progression and parameter adjustments over the training period:

- (a) **Learning Rate:** This graph shows the decay of the learning rate as training progresses, with a notable decrease over the global steps, reflecting the scheduled learning rate decay strategy applied to stabilize training.
- (b) **Global Step:** The global step chart shows the cumulative count of training steps. The incremental nature indicates steady progression through training batches.
- (c) **Epoch:** This plot displays the epoch count across training steps, showing a linear increase, as each epoch completes a pass over the dataset.
- (d) **Training Loss:** The training loss graph depicts a gradual decrease, indicating the model is minimizing its error on the training dataset over time.
- (e) **Gradient Norm:** The gradient norm plot shows the magnitude of gradients used in backpropagation. An increasing gradient norm may indicate that the model is encountering larger updates in later stages,

which can sometimes imply adjustments for optimization.

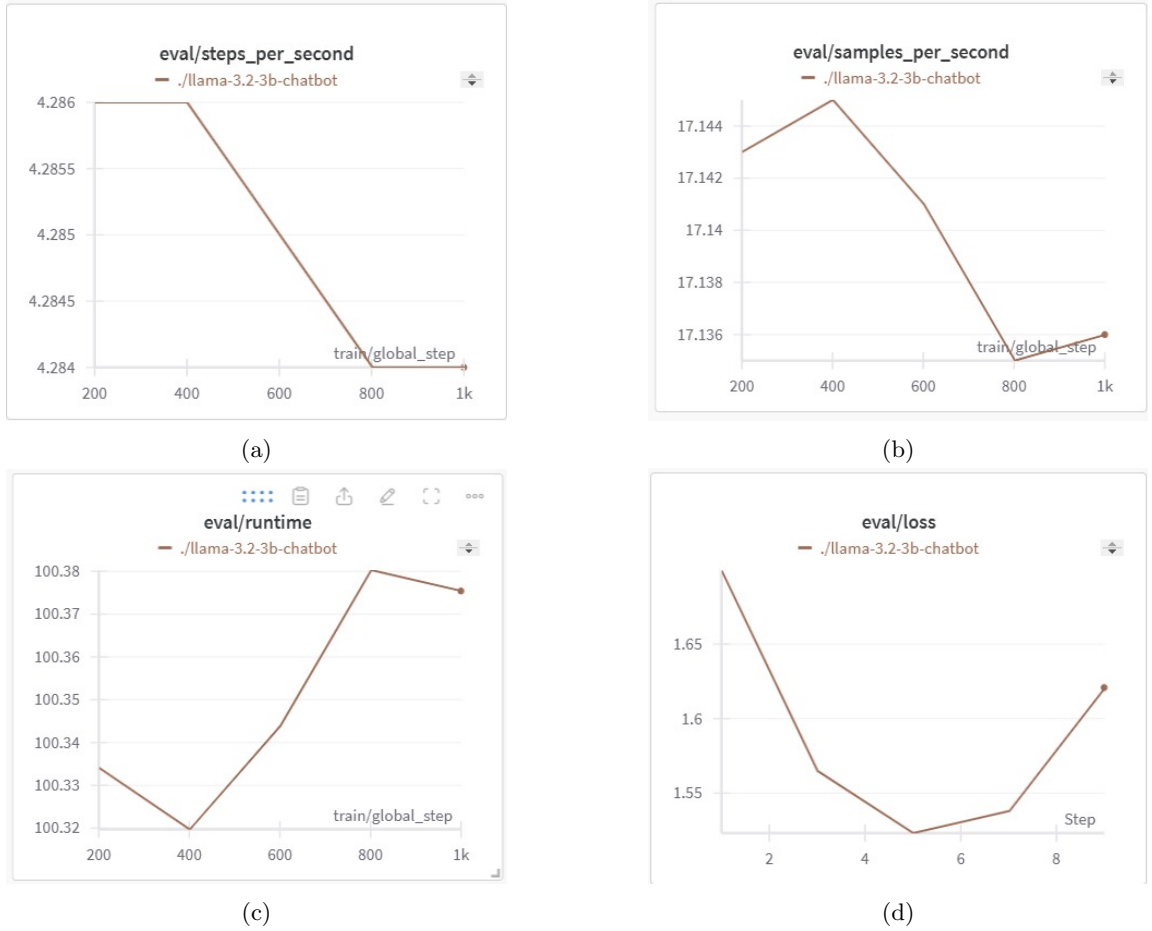


Figure 8: Evaluation Metrics of Fine-Tuned Model Across Training Steps

Description: The charts shown in figure 7 display key evaluation metrics for the fine-tuned financial chatbot model across various training steps. These metrics provide insight into the model's performance and efficiency during training:

- (a) **Steps per Second:** This chart shows the number of steps the model completes per second. We observe a slight decline in steps per second as training progresses, possibly due to increased computational demands at later stages.
- (b) **Samples per Second:** This metric reflects the number of samples processed per second. There is an initial increase, followed by a gradual decrease, suggesting variations in processing speed across training.
- (c) **Runtime:** The runtime plot indicates the total time taken at each evaluation step, showing a rising trend towards the end. This increase could be due to the complexity of the model's computations as it learns from the data.
- (d) **Loss:** This graph depicts the evaluation loss across different steps. After an initial decrease, the loss stabilizes before slightly increasing toward the final steps, indicating the model's performance fluctuated but generally improved over time.

Training Loss	Epoch	Step	Validation Loss
1.8392	0.9292	200	1.6995
1.445	1.8583	400	1.5649
1.1078	2.7875	600	1.5232
0.819	3.7166	800	1.5380
0.5895	4.6458	1000	1.6208

Figure 9: Training Loss and Validation Loss

Analysis of Training and Validation Loss

From figure 8, we can observe that after fine-tuning, the model's performance over a range of epochs and steps was tracked by monitoring the Training Loss and Validation Loss metrics. The observed results indicate a progressive decrease in both losses, signifying improved model fit on the training data and generalization capability on the validation set.

- **Training Loss:** Over the course of the fine-tuning process, the training loss consistently decreased, starting from 1.8392 at the initial steps and reducing to 0.5895 by step 1000. This indicates that the model became progressively better at fitting the training data.
- **Validation Loss:** The validation loss showed a similar initial decrease but started to level off toward the later stages, fluctuating slightly around the 1.52 - 1.62 range. This stabilization suggests that the model may have reached its optimal generalization capacity on the validation set, indicating a good balance between bias and variance.
- **Convergence:** The decreasing trend in training and validation losses suggests effective convergence of the model. However, the slight increase in validation loss at the final step (1.6208) might indicate a small degree of overfitting, which could be addressed by further regularization if required.

8 Plan for Novelty Assessment

(1) For the novelty assessment of this financial chatbot project, I propose integrating a Retrieval-Augmented Generation (RAG) pipeline with the existing fine-tuned model. This enhancement would allow the chatbot to provide more accurate and contextually relevant answers by combining static knowledge from the fine-tuned model with up-to-date information on current market conditions. By implementing the RAG architecture, the chatbot can retrieve real-time or recent market data in response to user queries and use this data alongside its fine-tuned knowledge base to generate answers that are both informed by historical insights and reflective of the latest financial trends. This approach not only improves the relevance and accuracy of responses but also differentiates the chatbot from typical static models, providing a competitive advantage in rapidly changing financial environments.

(2) In conventional language models, when the maximum token limit is reached during text generation, the response may be cut off mid-sentence, leading to incomplete or abrupt outputs. This disrupts user experience and reduces the perceived intelligence and usefulness of the chatbot. So we are planning to improve the chatbot’s response quality by implementing a feature that ensures the model completes responses with full meaning, even when approaching the token limit, similar to ChatGPT’s behavior.

9 Conclusion

The project aimed to develop a sophisticated chatbot capable of answering user queries on various aspects of trading in the Indian stock market. The chatbot was designed to handle a range of questions, from general inquiries about industry sectors to specific queries regarding individual companies. Additionally, the bot was expected to provide responses with evidence or reasoning, enhancing its utility in making informed trading decisions. This required the model to be updated with real-time data to accurately capture the impact of current events on the Indian stock market.

Methodology:

(1) Data Collection and Processing:

- **Web Scraping:** To ensure the chatbot’s responses were relevant and up-to-date, we performed extensive web scraping on financial news websites. This allowed us to collect recent and current-event-driven information pertinent to the Indian stock market.
- **Dataset Creation:** Using the collected articles, we constructed a dataset with question-answer-context pairs, enabling the model to relate questions to specific contexts in the stock market. We further enhanced the dataset by labeling and augmenting it where needed to improve the chatbot’s generalization and accuracy.

(2) Model Selection and Fine-Tuning:

- **Pretrained Model Choice:** The Meta LLaMA-3.2 3B INSTRUCT model was chosen as the base due to its ability to handle complex instruction-based tasks. Its architecture and pretraining made it suitable for fine-tuning on specific, domain-focused data.
- **LoRA (Low-Rank Adaptation) for Parameter Efficiency:** To manage memory constraints and improve training efficiency, we used LoRA, which enables efficient fine-tuning by adjusting a small number of parameters without modifying the core model.
- **Quantization Optimization:** Leveraging the BitsAndBytes library, we performed 4-bit quantization, which reduced memory usage without significant losses in model accuracy.

(3) Training Pipeline:

- **Data Splitting:** We split our dataset into training, validation, and test sets for robust evaluation.
- **Training Configurations:** Using the Hugging Face Trainer API, we set up training parameters, including batch sizes, learning rates, and gradient accumulation, to optimize model training.

- **Performance Tracking:** During training, evaluation was done periodically using evaluation loss, which allowed us to identify and save the best-performing model configuration.
- **Early Stopping:** We implemented an early stopping mechanism to prevent overfitting, ensuring that training halts when improvements plateau.

(4) Deployment and Interface:

- **Interface Design with Gradio:** We used Gradio to create a user-friendly interface, enabling users to interact with the chatbot effectively. Gradio’s capabilities helped in deploying a visually simple and functional interface for direct question-answering with the model.
- **Model Merging and Deployment:** To deploy the chatbot, we merged and offloaded the fine-tuned model, saving it for streamlined, inference-only use.

Results and Significance: The chatbot successfully answers user queries on the Indian stock market with a reasonable understanding of context and up-to-date insights. The use of LoRA and 4-bit quantization achieved memory efficiency without compromising the model’s ability to deliver accurate responses. Preliminary results indicate that the chatbot is effective in guiding users with actionable insights based on recent news events, market trends, and historical data.

Future Work:

- (1) To enhance novelty, I propose integrating a Retrieval-Augmented Generation (RAG) pipeline with the fine-tuned model. This will enable the chatbot to combine its trained knowledge with real-time market data, allowing it to answer questions based on both historical insights and current market conditions, thereby improving response relevance and accuracy.
- (2) Improve the chatbot’s response quality by implementing a feature that ensures the model completes responses with full meaning, even when approaching the token limit, similar to ChatGPT’s behavior.

References

- [1] Junjie Ye , Yuming Yang , Qi Zhang , Tao Gui , Xuanjing Huang , Peng Wang , Zhongchao Shi , Jianping Fan . Empirical Insights on Fine-Tuning Large Language Models for Question-Answering. Open-access paper on arXiv, 2024. (<https://arxiv.org/pdf/2409.15825>)
- [2] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLORA: Efficient Finetuning of Quantized LLMs . NeurIPS 2023 conference , 2023. (<https://openreview.net/pdf?id=OUIFPHEgJU>)
- [3] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothee Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, Guillaume Lample. LLaMA: Open and Efficient Foundation Language Models. Computing Research Repository (CoRR) under arXiv ID 2302.13971(available as an open-access paper on arXiv), 2023. (<https://arxiv.org/abs/2302.13971>)